

Práctica 7: Implementación de un codificador y decodificador de canal del estándar LTE

Tyrone Miguel Novillo Bravo
Facultad de Ingeniería
Departamento de Telecomunicaciones
Universidad de Cuenca
tyrone.novillo@ucuenca.edu.ec

Freddy Eduardo López Padilla
Facultad de Ingeniería
Departamento de Telecomunicaciones
Universidad de Cuenca
freddy.lopez@ucuenca.edu.ec

Resumen

En esta práctica se implementan en Python los dos códigos correctores de errores especificados por el estándar LTE: el código convolucional *tail-biting* de tasa 1/3, decodificado con el algoritmo de Viterbi de decisión suave, y el turbo código de tasa 1/3, formado por dos codificadores convolucionales recursivos sistemáticos concatenados en paralelo mediante un entrelazador QPP y decodificado de forma iterativa con dos decodificadores APP (BCJR). Ambos códigos se integran en una cadena OFDM sobre un canal multitrayecto Pedestrian A con ruido AWGN, y se evalúan transmitiendo texto y una imagen de prueba (Lena). Los resultados muestran la ganancia de codificación de ambos esquemas frente a la transmisión sin codificar: a 7 dB con 16-QAM el turbo código recupera el texto sin errores ($BER = 0.000$) mientras que sin codificación la BER es de 0.090. Como contraparte, se cuantifica el costo computacional: la decodificación turbo es en promedio $4.2\times$ más lenta que la de Viterbi, evidenciando el compromiso entre capacidad de corrección y tiempo de procesamiento.

Index Terms

LTE, codificación de canal, códigos convolucionales, turbo códigos, Viterbi, BCJR, QPP, BER, OFDM.

OBJETIVOS

- Implementar el codificador y el decodificador del código convolucional *tail-biting* de tasa 1/3 especificado en el estándar LTE.
- Implementar el codificador y el decodificador del turbo código de tasa 1/3 de LTE (dos codificadores RSC de 8 estados con entrelazador interno QPP).
- Integrar ambos códigos en una cadena de transmisión OFDM sobre un canal multitrayecto Pedestrian A y verificar su funcionamiento transmitiendo texto e imágenes.
- Comparar el desempeño de ambos códigos frente a la transmisión sin codificar mediante curvas de BER frente a SNR obtenidas por simulación de Monte Carlo.
- Medir y comparar el tiempo de ejecución de ambos esquemas de codificación y decodificación, y analizar el compromiso entre ganancia de codificación y costo computacional.

I. INTRODUCCIÓN

Los sistemas de comunicaciones inalámbricas operan sobre canales hostiles en los que el desvanecimiento multitrayecto y el ruido introducen errores en los bits transmitidos. Para prevenir y corregir estos errores, la capa física incorpora una etapa de *codificación de canal* que agrega redundancia controlada a la información: cada bit de datos se esparce en varios bits de código, de manera que el receptor pueda recuperar la información original incluso cuando parte de los bits llega corrompida [1]. El estándar LTE adopta dos códigos con este fin: un código convolucional *tail-biting* para los canales de control y un turbo código —piedra angular de su esquema de codificación— para los canales de datos [2].

En esta práctica se implementan desde cero, en Python, el codificador y el decodificador de ambos códigos, siguiendo las especificaciones del estándar y el material del curso [3]. Los códigos se integran en la cadena OFDM desarrollada en las prácticas anteriores (canal Pedestrian A [4] con ruido AWGN) y se evalúan de tres maneras: transmitiendo un texto de prueba (*lorem ipsum*), transmitiendo la imagen Lena, y mediante simulaciones de Monte Carlo que producen curvas de BER frente a SNR. Adicionalmente, se mide el tiempo de ejecución de cada esquema para cuantificar el costo computacional de la decodificación iterativa del turbo código frente a la decodificación de Viterbi del código convolucional.

II. MARCO TEÓRICO

II-A. Códigos convolucionales y decodificación de Viterbi

Un codificador convolucional es un sistema con memoria construido con registros de desplazamiento y sumadores módulo 2: cada bit de entrada produce n bits de salida que dependen del bit actual y de los m bits anteriores, con tasa de código $1/n$ [3]. LTE emplea un código de tasa 1/3 con $m = 6$ memorias (64 estados) y polinomios generadores $G_0 = 133_8$, $G_1 = 171_8$

y $G_2 = 165_8$. En lugar de agregar bits de cola, el estándar usa la técnica *tail-biting*: el registro se inicializa con los últimos 6 bits del bloque, de modo que el estado inicial y el final coinciden y no se desperdicia ancho de banda [2]. Al ser una máquina de estados finita, el código queda descrito por un diagrama de *trellis*, y la decodificación consiste en hallar la secuencia más probable que lo recorre; el algoritmo de Viterbi [5] resuelve esta búsqueda de forma óptima y eficiente.

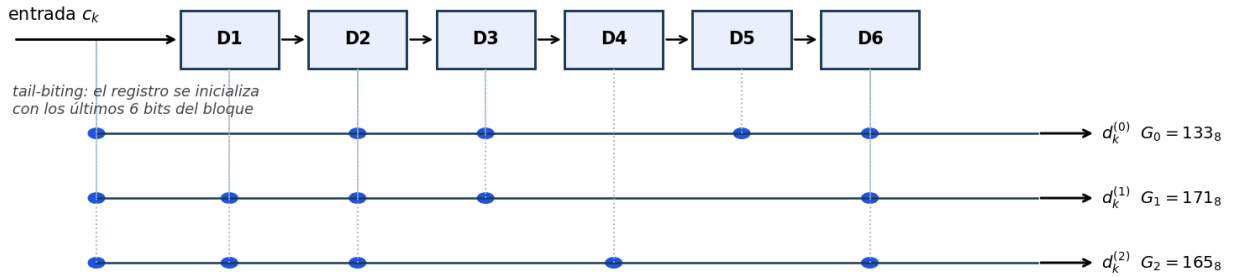
II-B. Turbo códigos y decodificación iterativa

Los turbo códigos [6] pertenecen a la categoría de codificación convolucional concatenada en paralelo (PCCC): dos codificadores convolucionales recursivos sistemáticos (RSC) idénticos, de 8 estados en LTE, procesan el mismo bloque de K bits, el primero en orden natural y el segundo tras un entrelazador interno. La salida son tres flujos: los bits sistemáticos S_k , la paridad $P1_k$ del primer RSC y la paridad $P2_k$ del segundo, lo que da una tasa ligeramente menor que $1/3$ por los bits de terminación de *trellis* [3]. El entrelazador de LTE es la permutación polinomial cuadrática (QPP) $\pi(i) = (f_1 i + f_2 i^2) \bmod K$, con coeficientes tabulados para 188 tamaños de bloque entre 40 y 6144 bits [2]. La decodificación es *iterativa*: dos decodificadores APP (*a posteriori probability*, algoritmo BCJR [7]) se pasan mutuamente decisiones suaves —información extrínseca— a través del entrelazador, y con cada iteración la estimación mejora, alcanzando un rendimiento cercano al límite de Shannon a costa de una complejidad computacional proporcional al número de iteraciones [1].

III. METODOLOGÍA – DESARROLLO DE LA PRÁCTICA

La implementación se realizó en Python con NumPy, reutilizando la cadena OFDM de las prácticas anteriores (modulación QAM, IFFT + prefijo cíclico, canal Pedestrian A, ecualización ZF). La Fig. 1 resume la estructura de los dos codificadores implementados.

(a) Codificador convolucional tail-biting LTE, tasa 1/3 (64 estados)



(b) Turbo codificador LTE (PCCC): 2 RSC de 8 estados + entrelazador QPP

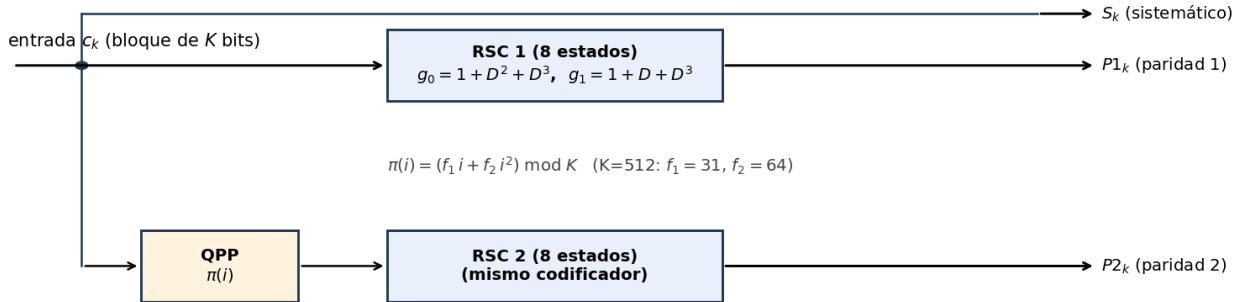


Figura 1. Esquema de los codificadores implementados: (a) convolucional *tail-biting* de tasa $1/3$ con sus tres polinomios generadores; (b) turbo codificador PCCC con dos RSC de 8 estados y entrelazador QPP.

III-A. Generación del código convolucional tail-biting

El codificador se implementó como una máquina de estados de 6 memorias. Cada generador octal se expande en 7 *taps* (entrada y memorias D_1 – D_6) y la salida de cada rama es la suma módulo 2 de los bits seleccionados por sus *taps*. La propiedad *tail-biting* se obtiene inicializando el registro con los últimos 6 bits del bloque, lo que garantiza que el estado final iguale al inicial sin transmitir bits de cola:

```

1 G_CONV = (0o133, 0o171, 0o165)      # polinomios generadores (octal)
2
3 def codificar_convolutacional(bits):
4     K = len(bits)
5     taps = _conv_taps()                # 7 taps por generador: [entrada, D1..D6]
6     # tail-biting: el registro arranca con los ULTIMOS 6 bits del bloque
7     estado = sum(int(bits[K - 1 - i]) << i for i in range(6))
8     d = np.empty((K, 3), dtype=np.uint8)
9     for k in range(K):
10         estado, d[k] = _conv_paso(estado, int(bits[k]), taps)
11     return d.reshape(-1)              # 3 bits de codigo por bit de entrada

```

donde `_conv_paso` calcula las tres salidas como XOR de los *taps* activos y desplaza el registro ($D_1 \leftarrow \text{entrada}$, $D_{i+1} \leftarrow D_i$).

III-B. Generación del turbo código

El elemento base es el codificador RSC de 8 estados, definido por la realimentación $g_0(D) = 1 + D^2 + D^3$ y la paridad $g_1(D) = 1 + D + D^3$. Su *trellis* se tabula recorriendo los 8 estados y las 2 entradas posibles:

```

1 def _rsc_trellis():
2     # g0 = 1 + D^2 + D^3 (realimentacion), g1 = 1 + D + D^3 (paridad)
3     for s in range(8):
4         m1, m2, m3 = s & 1, (s >> 1) & 1, (s >> 2) & 1
5         for u in range(2):
6             a = u ^ m2 ^ m3                # realimentacion
7             z = a ^ m1 ^ m3                # bit de paridad
8             ns = a | (m1 << 1) | (m2 << 2) # estado siguiente

```

El entrelazador QPP es una sola línea vectorizada; para el tamaño de bloque adoptado en la práctica, $K = 512$, el estándar define $f_1 = 31$ y $f_2 = 64$:

```

1 def qpp_interleaver(K):
2     f1, f2 = QPP[K]                      # K=512 -> (f1, f2) = (31, 64)
3     i = np.arange(K)
4     return (f1 * i + f2 * i * i) % K

```

El turbo codificador aplica el mismo RSC dos veces —al bloque en orden natural y al bloque entrelazado— y cada RSC se termina de forma independiente con 3 bits de cola que fuerzan el registro al estado cero, por lo que cada flujo tiene longitud $K + 4$ como indica el estándar:

```

1 def codificar_turbo(bits):
2     K = len(bits)
3     perm = qpp_interleaver(K)
4     z1, t1s, t1p = _rsc_codificar(bits, trellis)    # paridad 1
5     z2, t2s, t2p = _rsc_codificar(bits[perm], trellis) # paridad 2
6     # salida: sistematico + paridad 1 + paridad 2 + colas (tasa ~ 1/3)
7     return {"sys": bits, "par1": z1, "par2": z2, ...}

```

III-C. Recepción: demapeo suave y decodificadores

Tras la ecualización ZF, el demapeo no decide bits (decisión dura) sino que calcula un LLR (logaritmo de la razón de verosimilitud) por bit, ponderado por la calidad $|H[k]|^2$ de la subportadora que lo transportó: los bits que cayeron en un desvanecimiento entran al decodificador con poca confianza y los demás los corrigen. Este intercambio de decisiones suaves es el que habilita la ganancia de codificación [3].

El código convolutacional se decodifica con Viterbi de decisión suave sobre el *trellis* de 64 estados; como el *tail-biting* no fija el estado inicial ni el final, se recorre el bloque circular varias vueltas (4 en esta implementación) y se decodifica la última. El turbo código se decodifica con dos decodificadores APP (BCJR en su versión max-log) en lazo de realimentación durante 6 iteraciones:

```

1 def decodificar_turbo(Lsys, Lp1, Lp2, ..., perm, K, n_iter=6):
2     La1 = np.zeros(K)                    # informacion a priori inicial
3     for _ in range(n_iter):
4         Le1 = _bcjr_maxlog(Lsys, Lp1, La1, ...)    # APP 1
5         Le2 = _bcjr_maxlog(Lsys[perm], Lp2, Le1[perm], ...) # APP 2
6         La1 = Le2[invperm]                # extrínseca de vuelta al APP 1
7     L_total = Lsys + Le1 + La1            # canal + ambos extrínsecos
8     return (L_total < 0).astype(np.uint8) # decision final por signo

```

III-D. Escenario de simulación y medición de tiempos

La información se segmenta en bloques de $K = 512$ bits que se codifican, se modulan (QPSK o 16-QAM), se transmiten por la cadena OFDM y se decodifican por lotes. La Tabla I resume los parámetros. Como fuentes de datos se usaron el texto *lorem ipsum* de dos párrafos (8 208 bits) y la imagen Lena reescalada a 96×96 píxeles RGB (221 184 bits, 432 bloques). El tiempo de procesamiento de cada transmisión (codificación + OFDM + canal + decodificación) se midió con `time.perf_counter()`, y las curvas de BER y de tiempo frente a SNR se obtuvieron por Monte Carlo con 3 corridas independientes por punto e intervalos de confianza del 95 % (t de Student).

Cuadro I
PARÁMETROS DE LA SIMULACIÓN.

Parámetro	Valor
Ancho de banda / Δf / CP	5 MHz / 15 kHz / normal (4.7 μ s)
Subportadoras útiles / N_{FFT}	300 / 512
Canal	Pedestrian A (ITU-R M.1225), Rayleigh por <i>tap</i> + AWGN
Ecuación / demapeo	Zero-Forcing / LLR max-log ponderado por $ H ^2$
Modulaciones	QPSK, 16-QAM
Tamaño de bloque K	512 bits (QPP: $f_1 = 31$, $f_2 = 64$)
Código convolucional	tasa 1/3, $G = (133, 171, 165)_8$, <i>tail-biting</i> , Viterbi suave (4 vueltas)
Turbo código	tasa 1/3, 2 RSC de 8 estados, BCJR max-log, 6 iteraciones
Monte Carlo	SNR de 0 a 15 dB, 3 corridas por punto, IC 95 %

IV. EVALUACIÓN – ANÁLISIS DE RESULTADOS

IV-A. Transmisión de texto (*lorem ipsum*)

El texto de prueba se transmitió a 7 dB con 16-QAM usando los tres esquemas sobre realizaciones equivalentes del canal. Sin codificación el texto llega ilegible en varios tramos; el convolucional reduce los errores a cerca de la tercera parte y el turbo código lo recupera exactamente:

```

1 --- Original -----
2 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec sodales ipsum vel ante vulputate, id feugiat
   ligula imperdiet. [...]
3
4 --- Sin codificar (BER = 0.090, 5 ms) -----
5 ?orem`ipsqo dodmb ri4 amet,$c/.sectet?r ad)?m3ci/g ehmt. Dnnecaso?!me3 ) `sum r?l aotg wuh?u?at?$ i?$fauwh`
   t$lkG=,$`iip-Vtie4o ?mks$llcq?!?uBpi6,`P2u?um n/n mdbtIs Sed,0asctmsan |?nci$ent n)si. Y` hac0hebmte?
   Qe`platea d)ct?ms`n ?t [...]
6
7 --- Convolucional (BER = 0.030, 95 ms) -----
8 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Don>[??dales ipsum vel ante vulputate, id feugiat
   ligula imperdic????8is??cup??2rpis, pretium non lectus sed, accumsan tincidunt nisi. In hac habitasse
   platea dictumst. Ut [...]
9
10 --- Turbo (BER = 0.000, 176 ms) -----
11 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec sodales ipsum vel ante vulputate, id feugiat
   ligula imperdiet. Duis lacus turpis, pretium non lectus sed, accumsan tincidunt nisi. In hac habitasse
   platea dictumst. Ut [...]

```

Este resultado ilustra el propósito de la codificación de canal: con la misma energía por símbolo y el mismo canal, la redundancia de tasa 1/3 más la decisión suave convierten una transmisión inutilizable en una transmisión sin errores.

IV-B. Transmisión de imagen (*Lena*)

La Fig. 2 repite el experimento con la imagen Lena (96×96 RGB, 432 bloques de código) a 7 dB con 16-QAM. Sin codificación la imagen se recibe cubierta de ruido de sal y pimienta ($BER \approx 8,6 \times 10^{-2}$); el código convolucional reduce la BER en un factor de ~ 6 y el turbo código en un factor de ~ 27 , entregando la imagen prácticamente intacta.

Transmisión de Lena (96×96) — 16-QAM, SNR = 7 dB, canal Pedestrian A

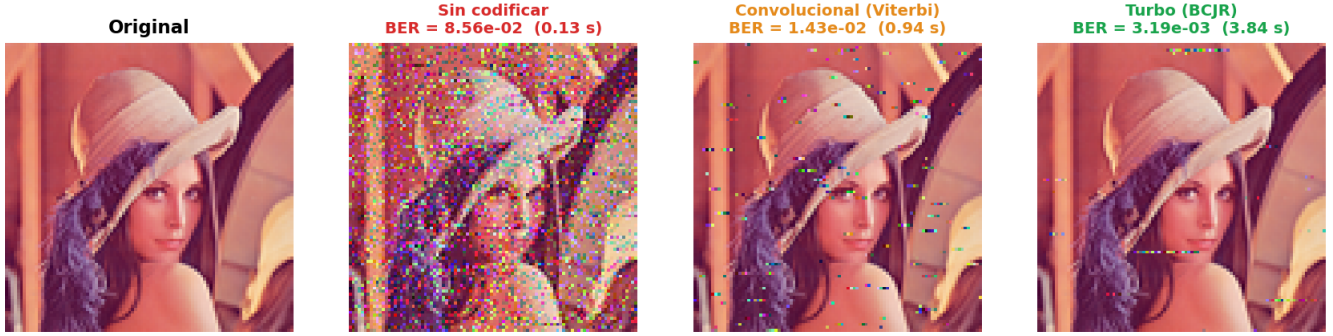


Figura 2. Recuperación de la imagen Lena con cada esquema de codificación (16-QAM, SNR = 7 dB, canal Pedestrian A). Bajo cada imagen se indica la BER y el tiempo de procesamiento de la transmisión.

IV-C. BER frente a SNR

La Fig. 3 muestra las curvas de BER frente a SNR obtenidas por Monte Carlo transmitiendo la imagen Lena completa en cada corrida (221 184 bits por corrida, 3 corridas por punto). Se observa el orden esperado $\text{turbo} \leq \text{convolutacional} \leq \text{sin codificar}$ en todo el barrido y para ambas modulaciones:

- El código convolutacional aporta una ganancia de codificación sostenida: a 8 dB con 16-QAM reduce la BER de $6,7 \times 10^{-2}$ a $9,9 \times 10^{-3}$.
- El turbo código exhibe el “acantilado” característico de la decodificación iterativa: con QPSK cae por debajo de 10^{-5} desde los 4 dB, y con 16-QAM se desploma entre 4 y 8 dB (de $2,7 \times 10^{-2}$ a $5,0 \times 10^{-4}$), alcanzando el piso de la simulación en 12 dB. Por debajo de ese umbral el intercambio iterativo de información extrínseca pierde su ventaja e incluso puede amplificar los errores (a 0 dB con 16-QAM su BER supera a la de la transmisión sin codificar), comportamiento típico de esta familia de códigos.
- Sin codificación la BER decae lentamente con la SNR, dominada por los desvanecimientos profundos del canal Pedestrian A: aún a 15 dB persisten errores.

BER vs SNR — códigos de canal LTE sobre OFDM + Pedestrian A (Monte Carlo, 3 corridas de 221 kbits, IC 95 %)

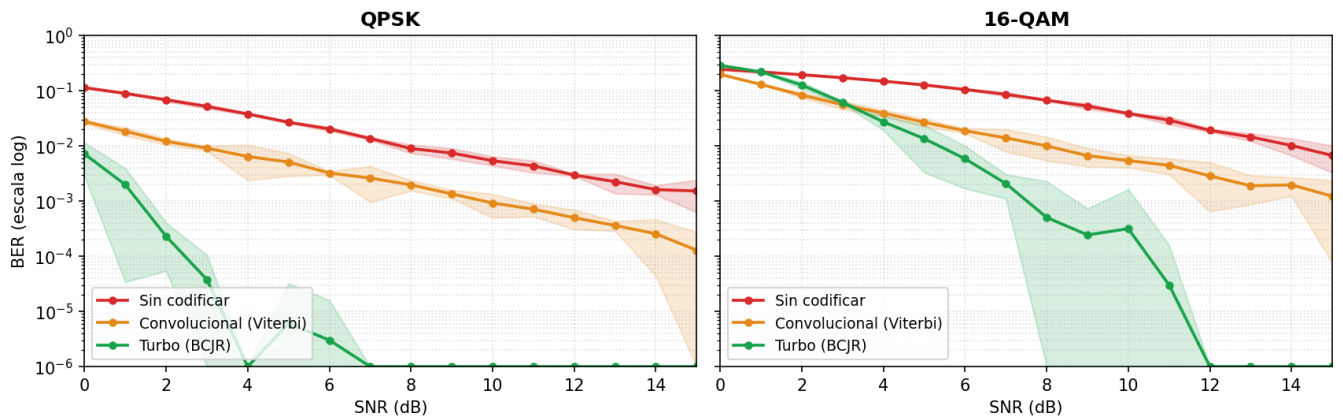


Figura 3. BER frente a SNR para los tres esquemas, con QPSK (izquierda) y 16-QAM (derecha). Sombreado: intervalo de confianza del 95 %.

IV-D. Tiempo de ejecución frente a SNR

La Fig. 4 presenta el tiempo de procesamiento de la transmisión completa en función de la SNR, y la Tabla II resume los promedios. Se destacan tres observaciones:

- El tiempo es esencialmente *independiente de la SNR* en los tres esquemas: tanto Viterbi como BCJR realizan la misma cantidad de operaciones sin importar cuán ruidosa llegue la señal (en esta implementación el número de iteraciones turbo es fijo).

- El turbo código es el más costoso: en promedio 3.84 s por transmisión con 16-QAM (8.9 ms por bloque de 512 bits), unas 4.2 veces el tiempo del convolucional (0.99 s). La diferencia proviene del decodificador: BCJR se ejecuta dos veces por iteración durante 6 iteraciones (12 pasadas por el *trellis*), mientras que Viterbi hace una única pasada.
- La transmisión sin codificar es la más rápida (0.14 s) pero, como muestran las Figs. 2 y 3, a costa de una BER inaceptable en este canal.

Tiempo de ejecución de la cadena vs SNR — transmisión de Lena (221 kbits, 432 bloques de K=512)

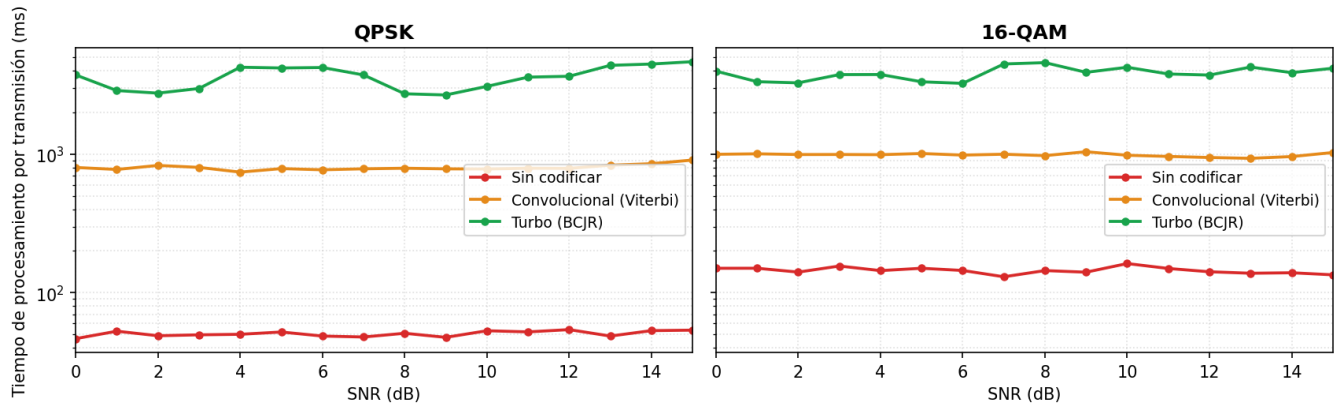


Figura 4. Tiempo de procesamiento por transmisión frente a SNR (escala logarítmica), con QPSK (izquierda) y 16-QAM (derecha).

Cuadro II

RESUMEN DEL COMPROMISO BER–TIEMPO (PROMEDIOS SOBRE EL BARRIDO DE SNR, TRANSMISIÓN DE 432 BLOQUES).

Esquema	QPSK			16-QAM		
	BER @ 4 dB	BER @ 8 dB	Tiempo (s)	BER @ 4 dB	BER @ 8 dB	Tiempo (s)
Sin codificar	$3,8 \times 10^{-2}$	$8,9 \times 10^{-3}$	0.05	$1,5 \times 10^{-1}$	$6,7 \times 10^{-2}$	0.14
Convolutacional	$6,4 \times 10^{-3}$	$2,0 \times 10^{-3}$	0.80	$3,9 \times 10^{-2}$	$9,9 \times 10^{-3}$	0.99
Turbo	$\leq 10^{-6}$	$\leq 10^{-6}$	3.61	$2,7 \times 10^{-2}$	$5,0 \times 10^{-4}$	3.84

El compromiso queda entonces cuantificado: el turbo código compra su rendimiento casi-Shannon con un decodificador $\sim 4.2\times$ más lento que Viterbi. Esto es consistente con el reparto de tareas del estándar: LTE reserva el turbo código para los canales de datos, donde el volumen de información justifica el costo, y usa el convolucional en los canales de control, cortos y sensibles a la latencia [1].

V. CONCLUSIONES

Se implementaron y verificaron el codificador y el decodificador de los dos códigos de canal del estándar LTE. El código convolucional *tail-biting* de tasa 1/3 quedó definido por sus tres polinomios generadores $(133, 171, 165)_8$ y su decodificación con Viterbi de decisión suave; el turbo código, por la concatenación en paralelo de dos RSC de 8 estados separados por el entrelazador QPP y su decodificación APP iterativa. La transmisión de texto y de la imagen Lena demostró de forma tangible la ganancia de codificación: en condiciones donde la transmisión sin codificar resulta inutilizable ($BER \sim 10^{-1}$ – 10^{-2}), el convolucional reduce la BER típicamente entre 3 y 7 veces y el turbo código recupera la información prácticamente sin errores.

Las curvas de Monte Carlo confirmaron el orden $\text{turbo} \leq \text{convolucional} \leq \text{sin codificar}$ en todo el barrido de SNR y para ambas modulaciones, con el “acantilado” característico del turbo código: por encima de un umbral de pocos dB la BER se desploma al piso de la simulación ($\leq 10^{-6}$), comportamiento que explica su elección como piedra angular de la codificación de datos en LTE.

La medición de tiempos cuantificó el precio de ese rendimiento: la decodificación turbo (2 pasadas BCJR $\times$ 6 iteraciones) resultó $\sim 4.2\times$ más lenta que la decodificación de Viterbi, y ambas son independientes de la SNR. Este compromiso rendimiento–complejidad se refleja directamente en el diseño del estándar, que asigna el turbo código a los canales de datos y el convolucional a los canales de control. Como trabajo futuro se podría incorporar el *rate matching* del estándar (perforado y versiones de redundancia HARQ) y una parada temprana de las iteraciones turbo, que reduciría el tiempo medio de decodificación en los puntos de SNR alta.

REFERENCIAS

- [1] E. Dahlman, S. Parkvall, and J. Sköld, *4G: LTE/LTE-Advanced for Mobile Broadband*, 2nd ed. Academic Press, 2014.
- [2] 3GPP, “Evolved universal terrestrial radio access (e-utra); multiplexing and channel coding,” 3rd Generation Partnership Project, Tech. Rep. TS 36.212, V10.7.0, 2012.
- [3] A. Vázquez-Rodas, *Comunicaciones Móviles — Capítulo 5: LTE, Codificación de Canal*, Cuenca, Ecuador, 2026.
- [4] ITU-R, “Guidelines for evaluation of radio transmission technologies for imt-2000,” International Telecommunication Union, Tech. Rep. Recommendation M.1225, 1997.
- [5] A. Viterbi, “Error bounds for convolutional codes and an asymptotically optimum decoding algorithm,” *IEEE Transactions on Information Theory*, vol. 13, no. 2, pp. 260–269, 1967.
- [6] C. Berrou, A. Glavieux, and P. Thitimajshima, “Near shannon limit error-correcting coding and decoding: Turbo-codes,” in *Proceedings of ICC '93 - IEEE International Conference on Communications*, vol. 2, 1993, pp. 1064–1070.
- [7] L. Bahl, J. Cocke, F. Jelinek, and J. Raviv, “Optimal decoding of linear codes for minimizing symbol error rate,” *IEEE Transactions on Information Theory*, vol. 20, no. 2, pp. 284–287, 1974.